

SSH Brute-Force Authentication Testing

Using Hydra Against Metasploitable2 in an Isolated Lab Environment

Prepared by:	Navya Bhatia
Attacker System:	Kali Linux (VirtualBox VM)
Target System:	Metasploitable2 (VirtualBox VM)
Network Mode:	VirtualBox Host-Only Adapter (isolated lab network)
Tool Used:	Hydra v9.7
Date:	July 4, 2026

Scope note: This exercise was performed entirely within a private, isolated virtual lab consisting of a Kali Linux attacker VM and a Metasploitable2 target VM, both of which are owned by the tester and intentionally configured to be vulnerable for educational purposes. No external or third-party systems were accessed.

1. Objective

The objective of this exercise was to demonstrate an SSH credential brute-force attack using the Hydra password-auditing tool against a deliberately vulnerable target machine (Metasploitable2), in order to understand how weak or default credentials can be exploited, and to reinforce best practices for securing SSH services.

2. Lab Environment

- Hypervisor: Oracle VirtualBox
- Attacker machine: Kali Linux (rolling release)
- Target machine: Metasploitable2 (intentionally vulnerable Linux distribution)
- Networking: Host-Only Adapter, placing both VMs on the 192.168.56.0/24 subnet
- Target IP: 192.168.56.106
- Target service: OpenSSH (port 22/tcp)

3. Methodology

3.1 Reconnaissance and Wordlist Preparation

The target's IP address was confirmed and an Nmap scan was run to verify that the SSH service was reachable and listening on port 22. Two input files were then created for Hydra: a username list (containing the known default account `msfadmin`) and a password wordlist containing a small set of common/default passwords, including the target's actual default password, to demonstrate a realistic weak-credential scenario.

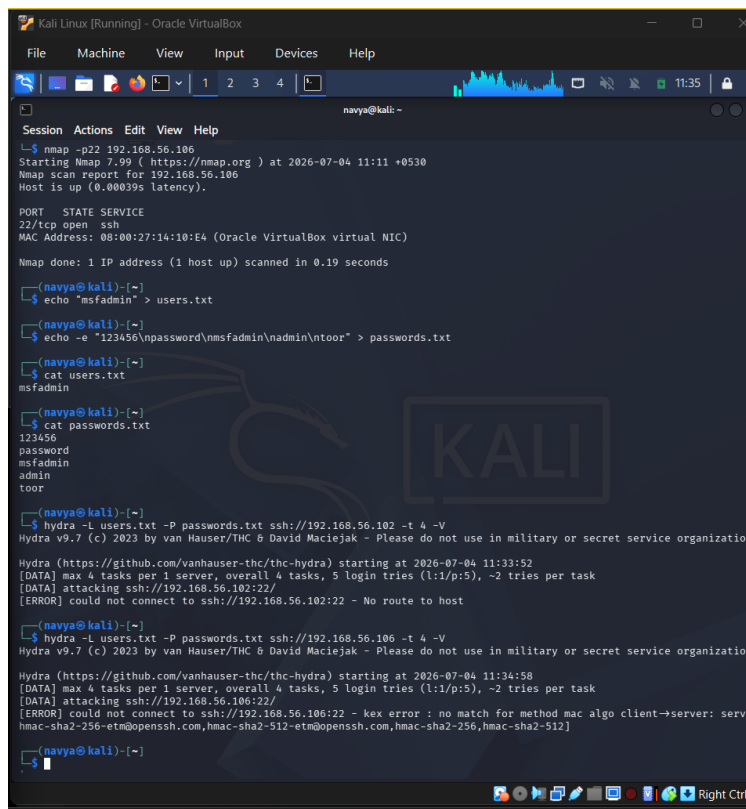
A screenshot of a Kali Linux terminal window running inside an Oracle VM VirtualBox. The terminal shows the following sequence of commands and outputs:
1. `nmap -p22 192.168.56.106`: An Nmap scan of port 22 on 192.168.56.106. The output indicates that the host is up and that port 22/tcp is open for the ssh service. The MAC address is 08:00:27:14:10:E4.
2. `echo "msfadmin" > users.txt`: A command to create a file named `users.txt` containing the username `msfadmin`.
3. `echo -e "123456\npassword\nmsfadmin\nadmin\ntoor" > passwords.txt`: A command to create a file named `passwords.txt` containing a list of passwords.
4. `cat users.txt`: A command to display the contents of `users.txt`, showing `msfadmin`.
5. `cat passwords.txt`: A command to display the contents of `passwords.txt`, showing the list of passwords.
6. `hydra -L users.txt -P passwords.txt ssh://192.168.56.102 -t 4 -v`: An initial Hydra brute-force attempt on the SSH service of 192.168.56.102. The output shows that it could not connect to the host.
7. `hydra -L users.txt -P passwords.txt ssh://192.168.56.106 -t 4 -v`: A second Hydra brute-force attempt on the SSH service of 192.168.56.106. The output shows a "kex error: no match for method mac algo client->server: serve" message, indicating a compatibility issue with the SSH client.

Figure 1: Nmap scan confirming SSH open on port 22, followed by wordlist creation (`users.txt`, `passwords.txt`) and the initial Hydra attempt.

3.2 Resolving SSH Compatibility Issues

Metasploitable2 runs a decade-old OpenSSH version that only supports legacy key-exchange, host-key, and MAC algorithms which modern SSH clients disable by default for security reasons. This initially caused connection failures (both for a manual SSH login test and for Hydra) with errors such as "no matching host key type found" and "kex error: no match for method". This was resolved by explicitly re-enabling the required legacy algorithms in the system-wide SSH client configuration file (`/etc/ssh/ssh_config`).

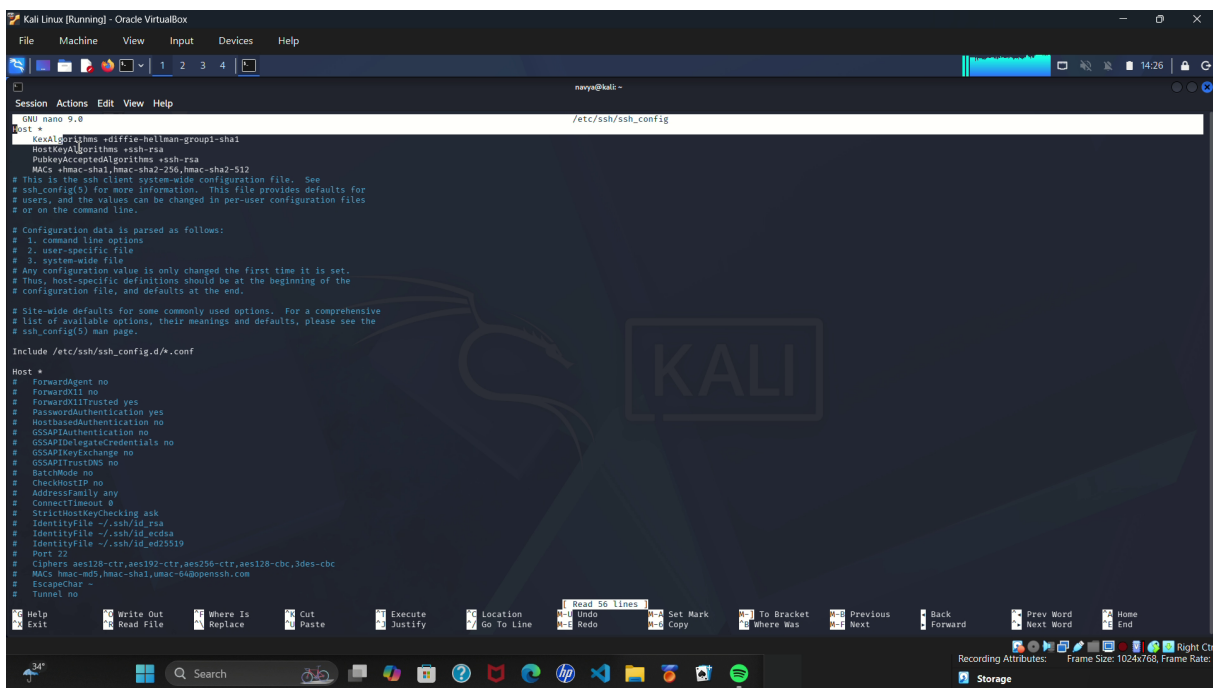


Figure 2: /etc/ssh/ssh_config updated to permit legacy KexAlgorithms, HostKeyAlgorithms, and MACs required to negotiate with Metasploitable2's outdated OpenSSH server.

3.3 Executing the Hydra Attack

With connectivity and protocol negotiation resolved, Hydra was run against the target's SSH service using the prepared username and password lists:

```
hydra -L users.txt -P passwords.txt ssh://192.168.56.106 -t 4 -V
```

The -V flag enabled verbose output so that each login attempt could be observed in real time, and -t 4 limited the attack to four parallel connections, an appropriate throttle for SSH brute-forcing.

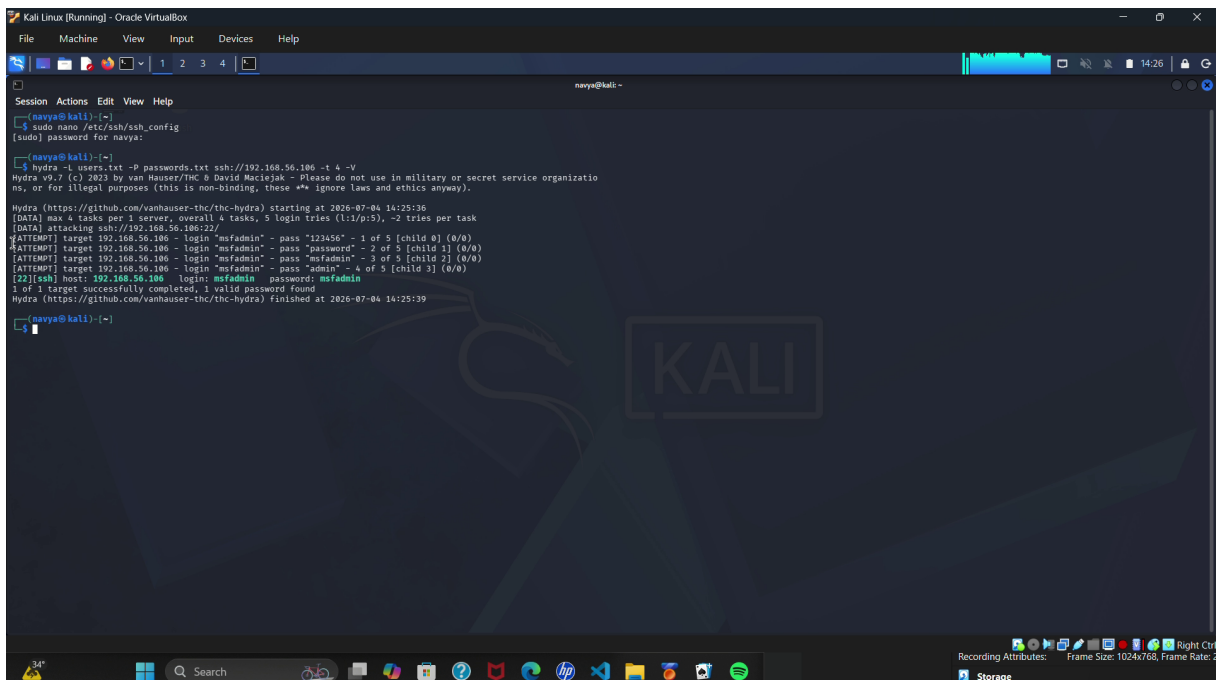


Figure 3: Hydra successfully identifying valid SSH credentials — login: msfadmin, password: msfadmin — after cycling through the supplied wordlist.

3.4 Verifying the Cracked Credentials

To confirm the credentials found by Hydra were genuinely valid, a manual SSH login was performed using the recovered username and password.

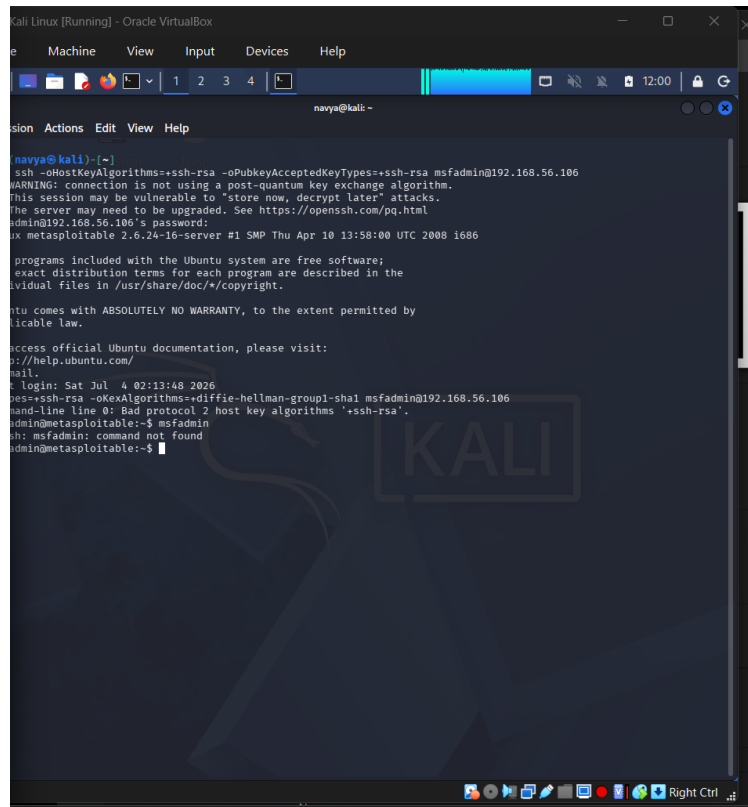


Figure 4: Manual SSH login to 192.168.56.106 using the cracked credentials (msfadmin/msfadmin), confirming successful authentication and shell access on the target.

4. Results

Target	192.168.56.106 (Metasploitable2)
Service	SSH (port 22)
Tool	Hydra v9.7
Username found	msfadmin
Password found	msfadmin
Outcome	1 of 1 target successfully completed — 1 valid password found

The weak, default credential pair `msfadmin:msfadmin` was successfully identified within a very small wordlist, demonstrating how quickly an SSH service can be compromised when default or weak

credentials remain in use.

5. Mitigation Recommendations

- Disable password-based authentication and enforce SSH key-based authentication only.
- Remove or change all default accounts and default credentials before deployment.
- Enforce strong, unique passwords via policy if password authentication must remain enabled.
- Deploy rate-limiting and lockout mechanisms such as fail2ban or denyhosts to throttle repeated failed logins.
- Keep OpenSSH and the underlying OS patched — this target was only vulnerable because it runs unmaintained, decade-old software.
- Restrict SSH access to trusted networks/IP ranges using a firewall, and consider changing the default port.
- Enable logging and alerting on repeated authentication failures to detect brute-force attempts in progress.

6. Conclusion

This exercise demonstrated the end-to-end process of an SSH brute-force attack in a controlled, isolated lab environment: reconnaissance, troubleshooting protocol compatibility, executing the attack with Hydra, and verifying the results. It highlighted how default or weak credentials combined with outdated software configurations can make a system trivially easy to compromise, and reinforced the importance of the mitigation controls outlined above. All testing was conducted solely against systems owned by the tester within a private virtual lab.